Clinical Cloud vs Edge Pipeline Benchmark

Updated technical report for the current Google Cloud Run + local edge benchmark: wearable alert validation, edge HTTP/TLS comparison, pipeline logs, metrics, and hospital dashboard outputs.

```
Repository: cloud-edge-pipeline-benchmark
Dataset: data/patients.json
Google Cloud project: benchmark-edge-cloud
Google Cloud region: europe-west8
Cloud Run service URL: https://benchmark-cloud-api-334241172753.europe-west8.run.app
Dashboard: http://localhost:8080
Benchmark command: docker compose --env-file .env.gcp run --rm benchmark
Scenarios: cloud, edge, edge_tls
Default run size: 20 patients x 3 scenarios x 3 repeats = 180 requests
```

1. What This Benchmark Demonstrates

The project now compares three request/response paths for the same synthetic patient wearable sample. The cloud path sends the sample to Google Cloud Run over platform HTTPS. The edge HTTP path sends the same sample to a local ward gateway without TLS. The edge TLS path sends it to the same local gateway over HTTPS/TLS. All paths execute the same wearable validation and out-of-range alert logic, then return an alert payload to the runner and dashboard.

The measurement intentionally excludes an edge-to-cloud callback. The benchmark is therefore an end-to-end latency comparison between the dashboard or runner and each processing destination, not a study of asynchronous data replication.

Important: the patients are synthetic. They are realistic enough for benchmarking and dashboard design, but they are not real patients and must not be interpreted as medical advice.

2. Patient Dataset

The benchmark uses 20 synthetic patient records. Each record contains demographics, location, diagnosis, comorbidities, medications, and three recent vital-sign samples.

```
patient_id, name, age, sex
ward, bed
primary_diagnosis
comorbidities[]
medications[]
vitals[]:
  timestamp
  heart_rate
  systolic_bp
  respiratory_rate
  spo2
  temperature
  glucose
```

Dataset snapshot

Patient	Ward / Bed	Diagnosis	Latest Vitals
P-0001 Anna Rossi (78F)	Cardiology C-12	Heart failure exacerbation	HR 108 | SBP 122 | RR 25 SpO2 92% | T 37.0 | Glu 118
P-0002 Marco Bianchi (64M)	Internal Medicine M-04	Community acquired pneumonia	HR 121 | SBP 108 | RR 30 SpO2 88% | T 38.7 | Glu 191
P-0003 Giulia Conti (42F)	Neurology N-09	Migraine observation	HR 76 | SBP 120 | RR 16 SpO2 99% | T 36.5 | Glu 91

Patient	Ward / Bed	Diagnosis	Latest Vitals
P-0004 Luca Ferri (70M)	Emergency E-02	Suspected sepsis	HR 135 | SBP 84 | RR 34 SpO2 89% | T 39.5 | Glu 155
P-0005 Sara Greco (55F)	Surgery S-18	Post-operative monitoring	HR 95 | SBP 121 | RR 20 SpO2 96% | T 37.5 | Glu 111
P-0006 Davide Romano (83M)	Geriatrics G-07	Dehydration and delirium	HR 115 | SBP 96 | RR 25 SpO2 94% | T 38.0 | Glu 218
P-0007 Elena Gallo (35F)	Obstetrics O-03	Postpartum observation	HR 86 | SBP 112 | RR 17 SpO2 98% | T 37.0 | Glu 92
P-0008 Paolo Ricci (59M)	Cardiology C-04	Acute coronary syndrome observation	HR 107 | SBP 140 | RR 22 SpO2 95% | T 36.8 | Glu 131
P-0009 Martina Riva (47F)	Oncology ON-11	Chemotherapy fever	HR 123 | SBP 98 | RR 26 SpO2 92% | T 39.1 | Glu 127
P-0010 Roberto Serra (68M)	Pulmonology P-15	COPD exacerbation	HR 114 | SBP 126 | RR 30 SpO2 86% | T 37.8 | Glu 140
P-0011 Chiara Lombardi (29F)	Emergency E-08	Syncope observation	HR 72 | SBP 110 | RR 15 SpO2 99% | T 36.5 | Glu 88
P-0012 Francesco Moretti (73M)	Nephrology K-06	Acute kidney injury	HR 101 | SBP 94 | RR 23 SpO2 95% | T 37.5 | Glu 180
P-0013 Valentina Neri (51F)	Internal Medicine M-14	Anemia workup	HR 90 | SBP 110 | RR 18 SpO2 97% | T 36.8 | Glu 101
P-0014 Stefano Costa (61M)	ICU Stepdown I-05	Post-sepsis recovery	HR 109 | SBP 106 | RR 26 SpO2 92% | T 38.1 | Glu 154
P-0015 Noemi De Luca (24F)	Surgery S-03	Appendectomy observation	HR 82 | SBP 116 | RR 16 SpO2 98% | T 37.0 | Glu 93
P-0016 Giorgio Fontana (80M)	Stroke Unit ST-10	Ischemic stroke monitoring	HR 102 | SBP 154 | RR 22 SpO2 95% | T 36.8 | Glu 145
P-0017 Irene Marchetti (66F)	Endocrinology D-02	Hyperglycemia	HR 95 | SBP 132 | RR 19 SpO2 97% | T 36.8 | Glu 280
P-0018 Alessio Vitale (57M)	Trauma T-06	Rib fracture monitoring	HR 108 | SBP 118 | RR 26 SpO2 92% | T 37.3 | Glu 108
P-0019 Federica Barbieri (72F)	Cardiology C-17	Atrial fibrillation rate control	HR 132 | SBP 120 | RR 23 SpO2 94% | T 37.0 | Glu 124
P-0020 Matteo Villa (39M)	Infectious Diseases ID-01	Viral syndrome	HR 96 | SBP 116 | RR 19 SpO2 97% | T 38.3 | Glu 103

3. Cloud Pipeline in Detail

The cloud pipeline is the only cloud execution path in the current stack. It receives the wearable sample at the public Google Cloud Run /process endpoint, validates the payload, checks the latest vital-sign sample against clinical thresholds, stores the full patient record for the cloud scenario, and returns the alert result to the caller.

```
dashboard / benchmark runner
  -> Google Cloud Run /process
  -> wearable payload validation
  -> out-of-range threshold checks
  -> alert generation
  -> cloud storage stage
  -> response to dashboard/runner
```

Information sent to Cloud Run

- Patient demographics and hospital location.
- Diagnosis, comorbidities, and medications.
- Wearable vital-sign series for heart rate, systolic blood pressure, respiratory rate, SpO2, temperature, and glucose.
- Benchmark metadata such as input_id, complexity, and security profile.

Information returned by cloud

- Risk score and alert level: low, medium, high, or critical.
- Out-of-range vital findings with value, unit, normal range, and direction.
- Recommended clinical action.
- Timing fields for network, validation, threshold check, storage, and total service time.
- Process age for the service instance that handled the request.

4. Edge Pipeline in Detail

The edge pipeline runs the same wearable alert validation locally in Docker. The edge is configured as a constrained ward gateway with 0.5 CPU, 256 MB memory, and a 128 process limit. The HTTP and TLS variants use identical application logic and resource limits so the benchmark can isolate local transport and request overhead.

```
dashboard / benchmark runner
  -> edge-api /process
  -> wearable payload validation
  -> out-of-range threshold checks
  -> local alert generation
  -> response to dashboard/runner
```

Information processed locally at the edge

- The same patient sample sent to Cloud Run.
- The latest wearable vital-sign sample.
- The configured normal ranges for each vital sign.
- A local alert summary with pseudonymized patient hash.

Information produced by the edge

```
patient_id
patient_hash pseudonymized from patient_id
ward
bed
latest_vitals
risk_score
risk_level
abnormal_vitals
recommended_action
```

```
alert
```

During the measured edge scenarios, CLOUD_SYNC_URL is empty. Edge results are returned to the benchmark runner and dashboard only; payload_synced_kb and sync_ms remain zero unless an explicit sync endpoint is configured.

The edge service time is intentionally higher than the cloud service time. The local edge containers are configured as ward-gateway-0.5vcpu-256mb with PREPROCESS_MS=58 and INFERENCE_MS=110, while the Cloud Run deployment uses PREPROCESS_MS=24, INFERENCE_MS=36, and STORAGE_MS=12. This makes the edge path represent constrained hospital hardware rather than a full cloud runtime.

5. Google Cloud Run Deployment

The cloud API is deployed to Google Cloud Run. The compose stack no longer starts a local cloud API by default; it starts the local edge services, dashboard, Prometheus, Grafana, and benchmark runner while CLOUD_URL points to Cloud Run. In the current configuration, the Google Cloud project is `benchmark-edge-cloud`, the selected European region is `europe-west8`, and the deployed Cloud Run endpoint is `https://benchmark-cloud-api-334241172753.europe-west8.run.app`.

```
Google Cloud
  Artifact Registry: europe-west8-docker.pkg.dev/benchmark-edge-cloud/benchmark/pipeline-api:latest
  Cloud Run service: benchmark-cloud-api
  Public HTTPS endpoint: https://benchmark-cloud-api-334241172753.europe-west8.run.app
  Runtime role: cloud
  Transport security label: platform_tls
  Default min instances: 1

Local machine / hospital edge
  edge-api-local:     http://localhost:8001
  edge-api-tls-local: https://localhost:8444
  benchmark-control:  http://localhost:8090
  hospital-dashboard: http://localhost:8080
  prometheus:         http://localhost:9090
  grafana:            http://localhost:3000
```

The deployment script uses MinInstances=1 by default to keep the cloud service warm for a stable edge/cloud comparison. Passing MinInstances=0 or updating the service to --min-instances 0 enables separate Cloud Run cold-start experiments, but the application benchmark does not expose a cold-start counter because a local first-request flag would not represent real infrastructure startup.

Earlier versions exposed a cold_start_candidate value based on the first /process request observed by a Python process. That value was removed from the API response, benchmark CSV, dashboard summary, Prometheus metrics, and report because it only marked a local first request and could be mistaken for the real Cloud Run service startup time.

Execution path

```
benchmark-runner local
  -> HTTPS request to Google Cloud Run /process for cloud

benchmark-runner local
  -> HTTP request to edge-api /process for edge

benchmark-runner local
  -> HTTPS request to edge-api-tls /process for edge_tls

prometheus local
  -> scrapes local edge-api /metrics
  -> scrapes local edge-api-tls /metrics
  -> does not scrape Cloud Run by default
```

- The cloud path includes real internet routing from the local machine to Google Cloud Run.
- The edge paths keep validation and alert generation local and do not include cloud synchronization in measured latency.
- Prometheus avoids Cloud Run by default so it does not keep the cloud instance warm during latency experiments.

- An optional docker-compose.cloud-metrics.yml overlay enables Cloud Run /metrics scraping when cloud application metrics are desired.
- The hospital dashboard continues to read benchmark result files from the local results directory.

6. Clinical Processing Logic

The processing logic is implemented in `services/pipeline_api/app/clinical.py`.

Step	Cloud	Edge
Schema validation	Checks required patient fields and vital samples.	Same validation before local processing.
Latest sample extraction	Uses the latest wearable sample from the received series.	Uses the same latest wearable sample.
Range normalization	Clamps values to broad plausible sensor ranges before comparison.	Same normalization.
Threshold check	Compares HR, SBP, RR, SpO2, temperature, and glucose against configured normal ranges.	Same threshold logic.
Alerting	Returns low, medium, high, or critical with recommended action.	Returns the same alert model locally.
Storage/sync	Runs cloud storage stage and reports payload_synced_kb as the sent payload.	No cloud sync during benchmark; payload_synced_kb is 0.

Vital thresholds

Vital	Normal range	Unit
heart_rate	50 - 110	bpm
systolic_bp	90 - 160	mmHg
respiratory_rate	10 - 24	/min
spo2	94 - 100	%
temperature	35.8 - 38.0	C
glucose	70 - 180	mg/dL

Alert model

- Critical: SpO2 below 90.
- High: three or more out-of-range vital signs.
- Medium: at least one out-of-range vital sign.
- Low: no validation errors and no out-of-range vital signs.
- Any schema validation error also raises the alert flag.

7. Executed Benchmark Pipelines

```
cloud
edge
edge_tls
```

The default BENCHMARK_SCENARIOS value is cloud,edge,edge_tls. The runner repeats the 20-patient dataset three times and sends the three scenario requests for each patient concurrently, producing 180 rows in results.csv.

Scenario	Processing	Transport	Purpose
cloud	Cloud Run wearable validation	Google platform HTTPS	Measures client-to-Cloud-Run round trip plus service processing.
edge	Local wearable validation	HTTP	Measures constrained local ward-gateway processing without TLS.
edge_tls	Local wearable validation	HTTPS/TLS	Measures the same local edge application over TLS.

Latency interpretation

- Mean round trip is measured by the benchmark client and includes transport, HTTP/TLS overhead, service execution, and scheduling effects.

- Mean service in the dashboard is computed from the internal processing stages: preprocess, inference, storage, and sync.
- The edge and edge_tls service means are expected to be close to each other because real TLS overhead is mostly outside the internal service-stage sum.
- The dashboard no longer shows cold starts. Real Cloud Run cold-start analysis should be handled as a separate infrastructure experiment with min-instances=0 and Cloud Monitoring or controlled request timing.

8. Benchmark Outputs

```
results/results.csv
results/summary.json
results/patient_results.json
```

`results.csv` is the technical benchmark table. `summary.json` aggregates latency, payload, and alert counts by scenario. `patient_results.json` is optimized for the hospital dashboard.

Field	Meaning
repeat	Dataset repeat index. Default repeats are 1, 2, and 3.
patient_id, ward, bed, diagnosis	Clinical context of the processed patient.
client_total_ms	End-to-end latency seen by the benchmark client.
service_total_ms	Time measured inside the API handler.
client_service_delta_ms	Client-side overhead not measured inside the service. TLS handshake and HTTP overhead mainly appear here.
transport_security	plain, tls, or platform_tls.
process_age_ms	Age of the service process when it handled the request.
network_ms	Configured network delay stage.
preprocess_ms	Wearable payload validation stage.
inference_ms	Out-of-range threshold check and alert generation stage.
sync_ms	Cloud synchronization time. It is 0 for edge scenarios in the direct comparison.
payload_sent_kb	Input payload sent by the benchmark client.
payload_synced_kb	Payload stored or synchronized to cloud. Cloud reports the sent payload; edge reports 0 with sync disabled.
abnormal_vitals_count, abnormal_vitals	Number and details of out-of-range vital signs.
risk_score, risk_level	Computed patient risk.
recommended_action	Operational recommendation for the ward team.

9. Hospital Dashboard

The hospital dashboard is served by `services/hospital_dashboard/app/main.py` at `http://localhost:8080`.

- Patient worklist sorted by risk.
- Ward and risk filters.
- Latest vital signs for the selected patient.
- Diagnosis, medications, comorbidities, risk score, out-of-range values, and recommended action.
- Per-patient comparison of cloud, edge, and edge_tls latency, service time, transport, payload, and risk result.
- Latency table with runs, mean round trip, p99 round trip, and mean service for each scenario.
- No cold-start column: the previous local first-request marker was removed because it did not measure true infrastructure startup.
- Ward load and active alert count.
- Critical-patient dialog after a benchmark run when critical results are present.

The dashboard reads patient_results.json after the benchmark runs. If the benchmark has not been executed yet, it still displays the dataset and marks patients as pending.

10. Prometheus and Grafana

Prometheus collects runtime metrics from the API services. It does not execute the benchmark. Grafana visualizes those metrics over time. The hospital dashboard is separate: it visualizes clinical benchmark results.

```
pipeline_requests_total
pipeline_total_latency_ms
pipeline_stage_latency_ms
pipeline_payload_size_kb
pipeline_in_flight_requests
pipeline_process_started_at_seconds
```

The default Prometheus configuration scrapes only the local edge services. This avoids accidental Cloud Run warm-up before latency tests. If cloud application metrics are needed, configure-gcp-hybrid.ps1 also generates prometheus.gcp.with-cloud.yml and docker-compose.cloud-metrics.yml can mount it.

Structured pipeline logs

Both Cloud Run and local edge services emit JSON log events named clinical_pipeline_step. They include pseudonymized patient hash, ward, bed, diagnosis, vital-sample count, clinical fields used, timing, risk result, abnormal vital details, and process age. Patient names and raw vital values are not logged in the step context.

```
request_received
network_uplink
transport_security
wearable_payload_validation
abnormal_threshold_check
cloud_storage
dashboard_response
```

11. Commands

```
powershell -ExecutionPolicy Bypass -File scripts\deploy-cloud-run.ps1 `
  -ProjectId benchmark-edge-cloud `
  -Region europe-west8

start_benchmark.bat

docker compose --env-file .env.gcp up --build

docker compose --env-file .env.gcp run --rm benchmark

powershell -ExecutionPolicy Bypass -File scripts\deploy-cloud-run.ps1 `
  -ProjectId benchmark-edge-cloud `
  -Region europe-west8 `
  -MinInstances 0

docker compose ps
docker compose logs -f edge-api edge-api-tls benchmark-api hospital-dashboard
docker compose down
```

12. Limitations

- The dataset is synthetic and small by design.
- The alert model is deterministic and threshold-based; it is not a validated medical device or clinical decision support model.
- The edge is emulated through Docker CPU/RAM limits, not real bedside hardware.
- The edge TLS certificate is local and self-signed for benchmarking, not production use.
- Cloud Run HTTPS is real but terminated by the Google Cloud platform rather than the application container.
- The benchmark no longer reports cold starts because a process-local first-request marker would be a misleading proxy for real service startup.
- Real Cloud Run cold-start experiments require separate control of min-instances, warm-up traffic, monitoring scrapes, and request timing.
- The cloud benchmark depends on the local network path from the workstation to Google Cloud Europe.

- Network delay is simulated at application level; a stronger network study could use tc netem, Mininet, or Containernet.